

第3回

「大容量データ(画像)の処理」
ーメモリアクセスと処理時間ー

3次元配列の比較

●宣言 X型 aaax[L][M][N];

・要素 aaax[i][j][k] へのアクセス: アドレス計算1回、メモリアクセス2回

$$*(aaax + (i \times M \times \text{sizeof}(X) + j \times \text{sizeof}(X)) \times N + k \times \text{sizeof}(X))$$

・メモリ使用量 $L \times M \times N \times \text{sizeof}(X)$ バイト

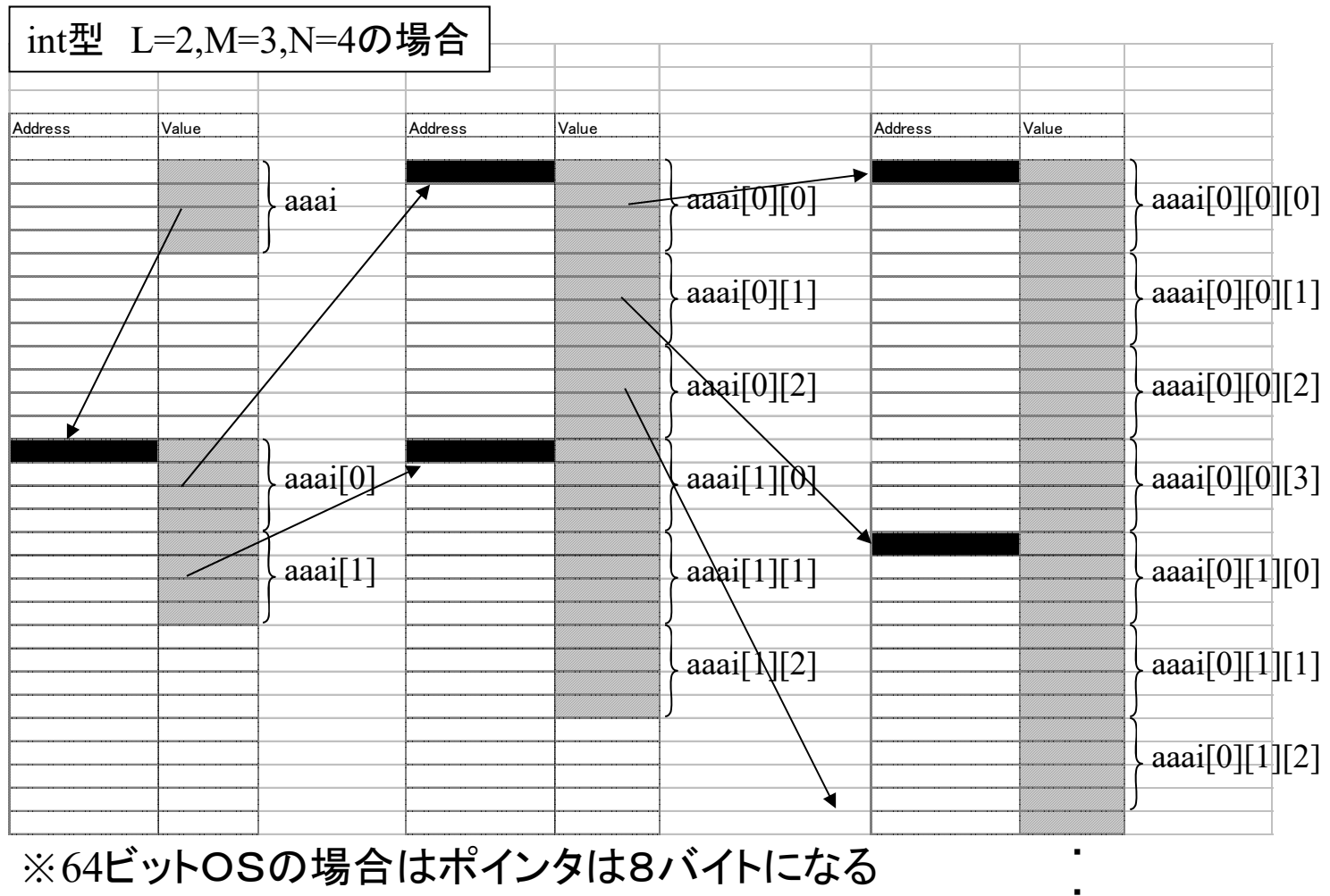
●動的確保(サイズ: $L \times M \times N$)

・要素 aaax[i][j][k] へのアクセス: アドレス計算3回、メモリアクセス4回

$$*((* (aaax + i \times \text{sizeof}(X **)) + j \times \text{sizeof}(X *)) + k \times \text{sizeof}(X))$$

・メモリ使用量 $L \times M \times N \times \text{sizeof}(X)$
 $+ L \times M \times \text{sizeof}(X *) + L \times \text{sizeof}(X **) + \text{sizeof}(X ***)$ バイト

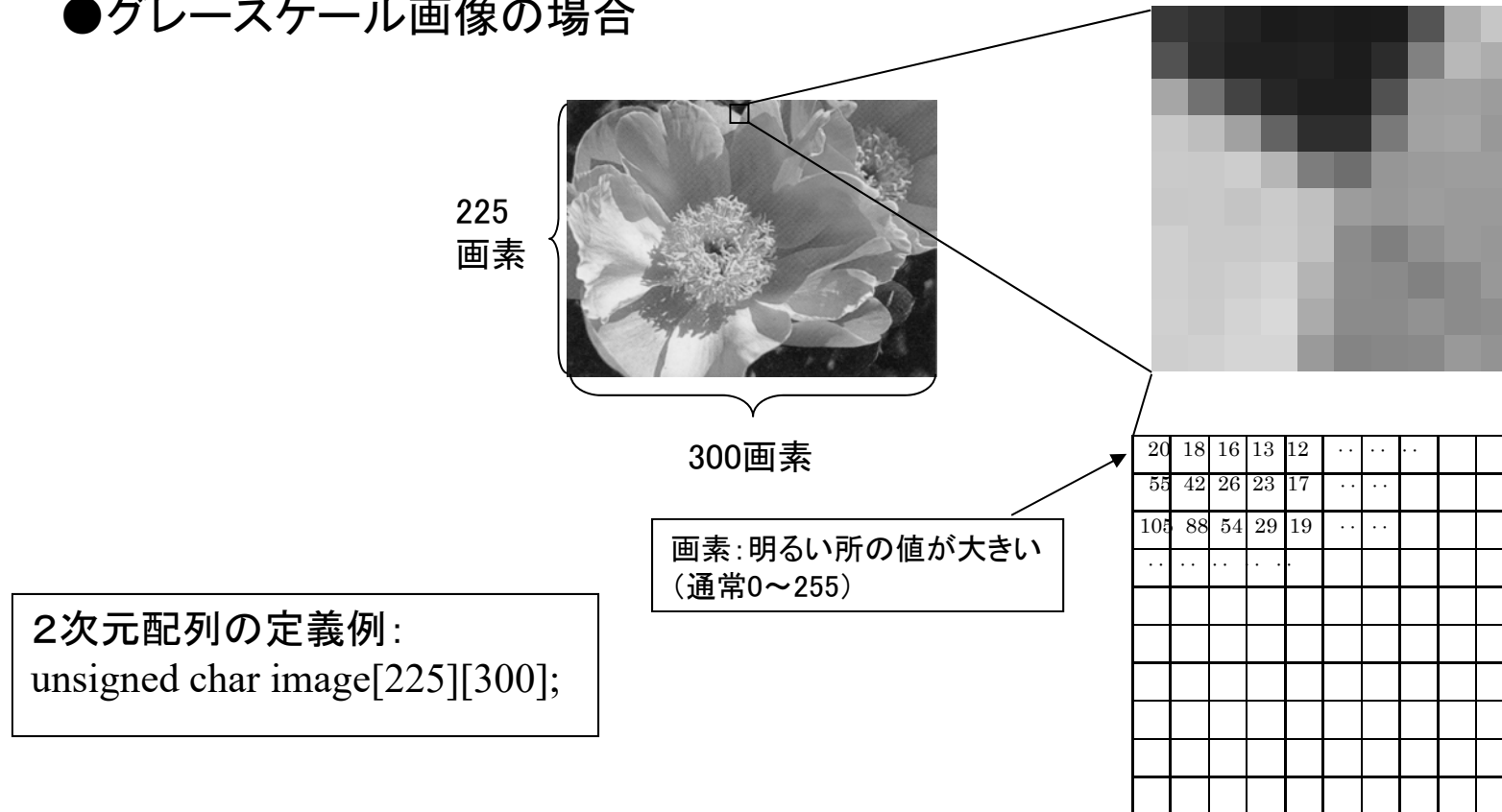
動的確保による3次元配列



多次元配列の用途(1)

- ・表、3次元データなど
- ・画像(グレースケール、カラー、マルチスペクトル)

●グレースケール画像の場合



2次元配列の定義例:
 unsigned char image[225][300];

多次元配列の用途(2)

●カラー画像の場合

1画素に3つの値
(R,G,B)を持つ

R=20 G=45 B=90			

3次元配列の定義例:

```
unsigned char image[3][225][300];
```

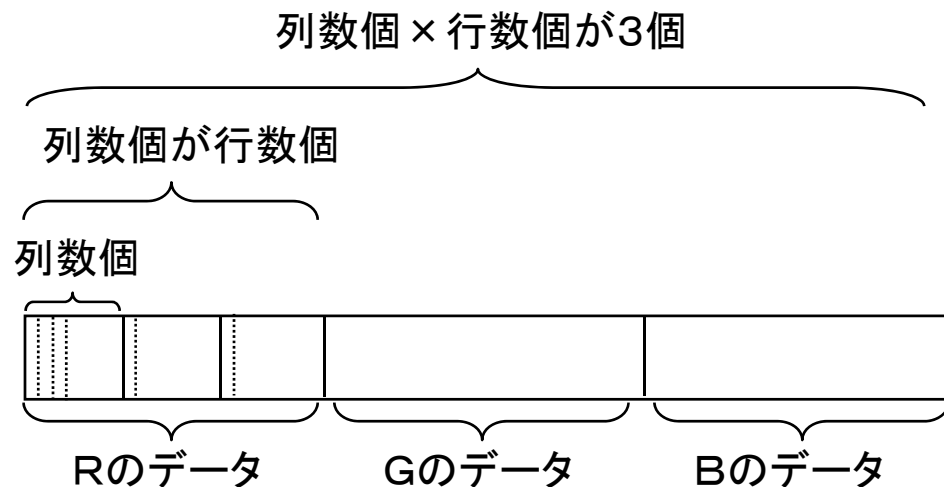
あるいは

```
unsigned char image[225][300][3];
```

画像データのメモリ内配置(1)

● `unsigned char image[3][行数][列数];` の場合

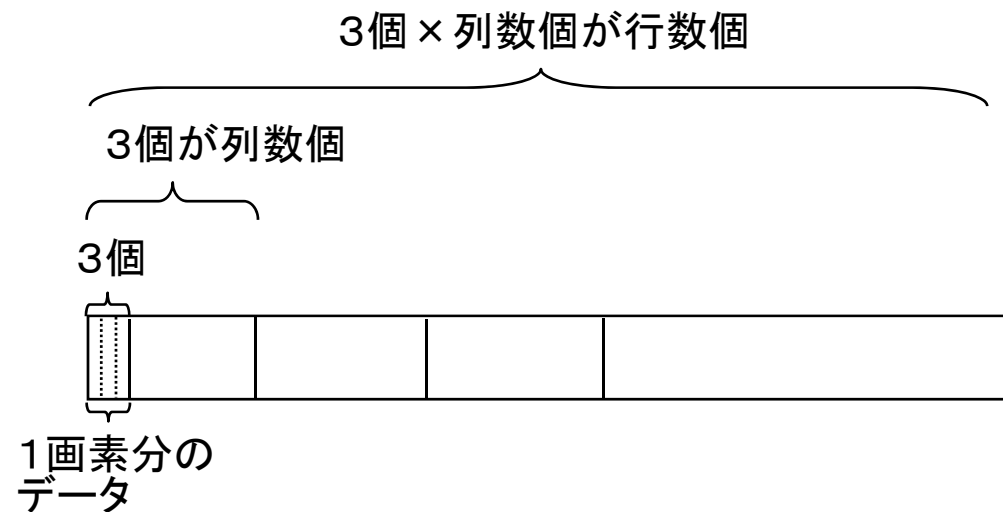
- ・R, G, Bのデータがそれぞれまとまっている。
- ・**CSQ** (Channel Sequential) 形式と呼ばれる。
- ・`image[i]` が1色分のデータを表し、グレースケール画像を引数とする関数に渡せる。



画像データのメモリ内配置(2)

● `unsigned char image[行数][列数][3];` の場合

- ・ 1画素内のRGBのデータが連続している。
- ・ **CIP** (Channel Interleaved by Pixel) 形式と呼ばれる。
- ・ `image[i][j]` が1画素分のデータを表し、1画素分のデータを処理する関数に渡せる。



画像の作成、コピー、出力実験

● kadai3-1.c

機能: カラー画像を作成し、MAX_REPEAT回コピーした後、PPMファイルとして出力

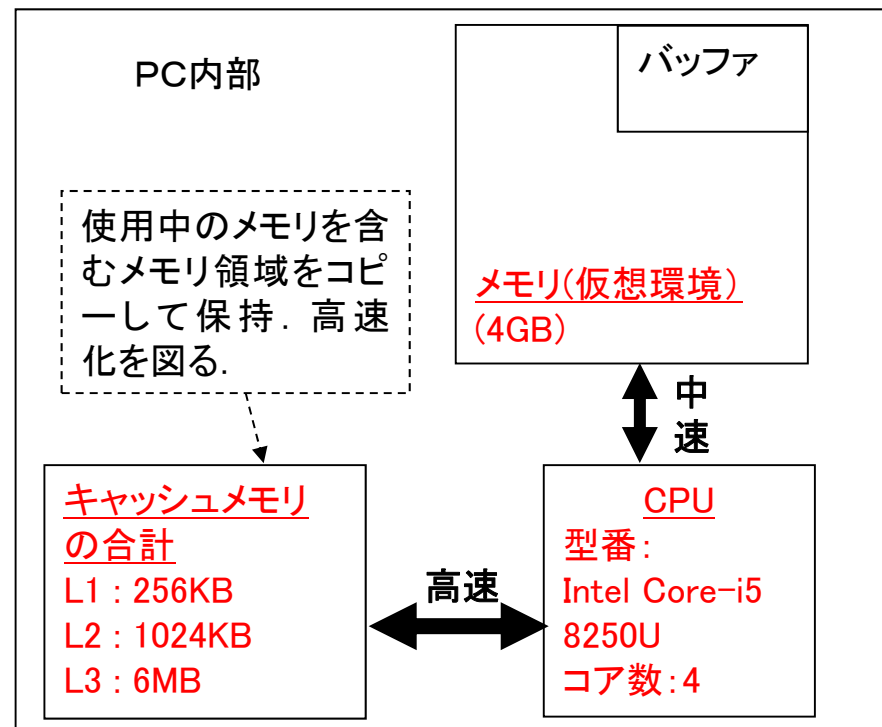
▲課題: CSQ、CIP形式のそれぞれについて実行し、出力された画像を表示する。

- ・式を変更し、オリジナルな画像を作成する.
- ・gcc -Wall -O2 -DCSQ kadai3-1.c -lm でコンパイルするとCSQ形式となる。

○PPMファイルについて

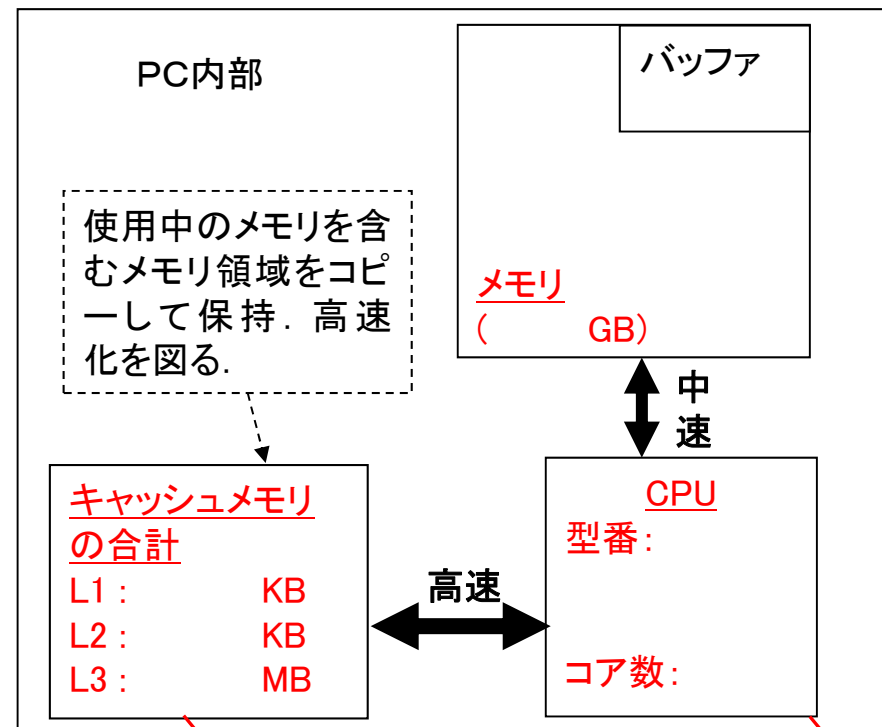
- ・display tmp.ppm で表示
- ・convert tmp.ppm tmp.??? で???形式に変換(??? : bmp, ps, jpgなど)
- ・Windowsの場合はPhotoshopなどが対応している

自分のPCのスペックを調べる (以下はノートPC CF-SVの例)



自分のPCのスペックを調べる

Windowsの場合は「タスクマネージャーのパフォーマンス」利用可
lscpu, lsmemでも調査可能.



L1, L2, L3の合計サイズを回答

CPUの型番を回答

コア数とコア当りのキャッシュメモリ (CF-SVの例)

- コア数:4
- L1キャッシュ:データ用は半分(残り半分は命令用)
 - コア当りのデータ用L1キャッシュ= $256\text{KB}/2/4=\underline{32\text{KB}}$
- L2キャッシュ:データと命令で共用
 - コア当りのL2キャッシュ= $1024\text{KB}/4=\underline{256\text{KB}}$
- L3キャッシュ:全てのコアで共用
 - L3キャッシュ=6MB

コピー時間測定実験(1)

▲課題: 画像サイズ、メモリ格納形式を変えてコピー時間を測定し、考察せよ。

- ・画像サイズ(SIZE): 5, 10, 20, 50, 100, 200, 500, 1000, 2000, 5000
- ・メモリ格納形式: CIP、CSQの両方について
- ・`res[i][j][k]=org[i][j][k]/2;` と変更し、値を2で割ってコピーする。(copy3dimg()内)
- ・測定は画像の出力を必ずコメントアウトしてから行う。
- ・コピー回数1000回あたりのコピー時間を測定する。
- ・消費時間が1秒～20秒程度の範囲になるようにMAX_REPEATを適当に変えて測定し、1000回当りの消費時間に換算する。
- ・以下でコンパイルする。
gcc -Wall -O2 -ftree-vectorize -fopt-info-vec -DCSQ kadai3-1.c -lm
- ・測定方法: `time a.out` の結果のうち、消費時間(real)を測定する。
(他の処理を極力行わない。)
- ・測定結果のバラツキが大きい場合は数回の平均値を求める。
- ・測定結果は両対数のグラフにする。
- ・最低限考察すべきこと: 両者の差の原因, 理論値との差の原因など

コピー時間測定実験(2)

▲課題: 以下のようにコピー時の「ループの順番を入れ替えた場合」について, 実験(1)と同様にコピー時間を測定して考察せよ.

- ・関数copy3dimg()内のfor文のうち, 最も内側(変数k)のfor文を一番外側に移動し, (k, i, j)の順番とする.
- ・実験(1)の順番(i,j,k)をCASE1, 実験(2)の順番(k,i,j)をCASE2と呼ぶことにする.
- ・測定結果は実験(1)と同様に両対数のグラフにする.
- ・グラフは, CIPの場合についての実験(1)と(2)の結果で1枚, CSQの場合について1枚の最低2枚作成して考察する.
- ・最低限考察すべきこと: 実験(1)と(2)の結果の差の原因, 理論値との差の原因など

SIZEが1000の時の以下の消費時間を回答せよ.

(1) CIP, CASE1

(2) CSQ, CASE2

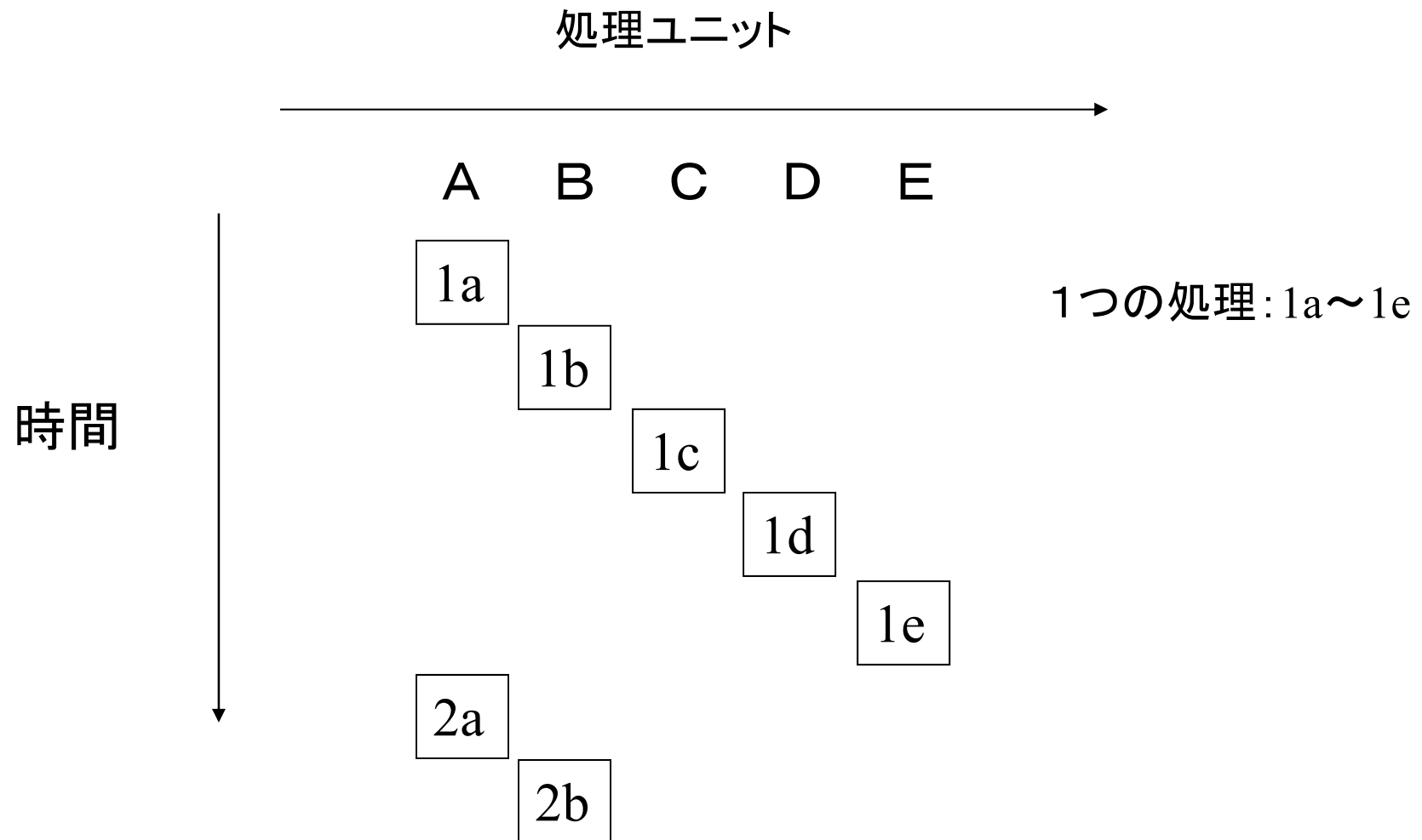
高速化に関わる仕組みの例

- パイプライン処理
- ベクトル化(SIMD化)
- キャッシュメモリ
- ループアンローリング

※処理の高速化には、CPUとコンパイラの特徴を把握して、高速化されやすいプログラムを書くことが重要

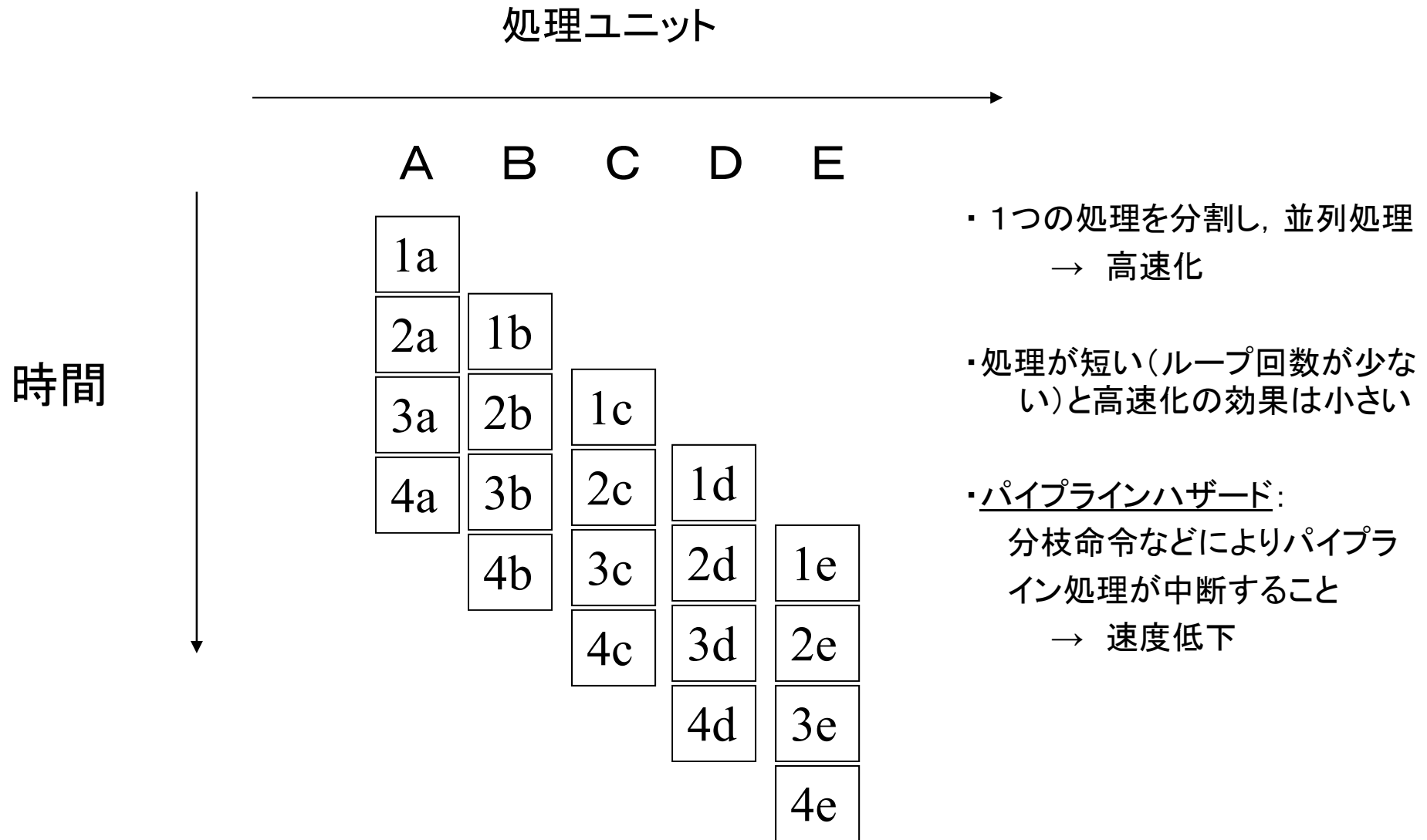
パイプライン処理(1)

●非パイプライン処理



パイプライン処理(2)

●パイプライン処理



ベクトル化 (SIMD化)

- 複数のデータをまとめる (ベクトル化)
- まとめたデータ (ベクトル) を一度に処理する
- 例: 配列の各要素に同じ処理をする場合, 複数の要素をNバイトにまとめて一度に処理 (NはCPUに依る. CF-SVではN=16.)
- Single Instruction Multi Data (SIMD)
- 自動ベクトル化のオプションは“-ftree-vectorize”
- “-O2”だけでは自動ベクトル化されない場合あり
- “-fopt-info-vec”でベクトル化された箇所が出力される

キャッシュメモリ(1)

- メインメモリよりも高速なメモリ
- 処理に必要なデータをメインメモリからコピーして保存しておき, 処理を高速化
- 高速な順番: $L1 > L2 > L3$
- サイズの順番: $L3 > L2 > L1$
- 管理はライン単位(16バイト~256バイト)

キャッシュメモリ(2)

- 必要なデータがキャッシュに存在 → キャッシュヒット
- キャッシュに存在しない → キャッシュミス
 - L1に存在しないならL2からコピー
 - L3にも存在しないならメインメモリからコピー
- キャッシュヒット率 = $\text{ヒット数} / (\text{ヒット数} + \text{ミス数})$
- キャッシュミスの場合, キャッシュに持ってくる新たなデータをキャッシュのどのデータに上書きするか?
 - アクセスされたのが最も古いラインに上書きする場合が多い. LRU方式 (Least Recently Used) という.

ループアンローリング(ループ展開)

例:

ループ展開前

```
for(j=0;j<DIM2;j++)  
  for(k=0;k<3;k++)  
    res[j][k]=org[j][k];
```

ループ展開後

```
for(j=0;j<DIM2;j++){  
  res[j][0]=org[j][0];  
  res[j][1]=org[j][1];  
  res[j][2]=org[j][2];  
}
```

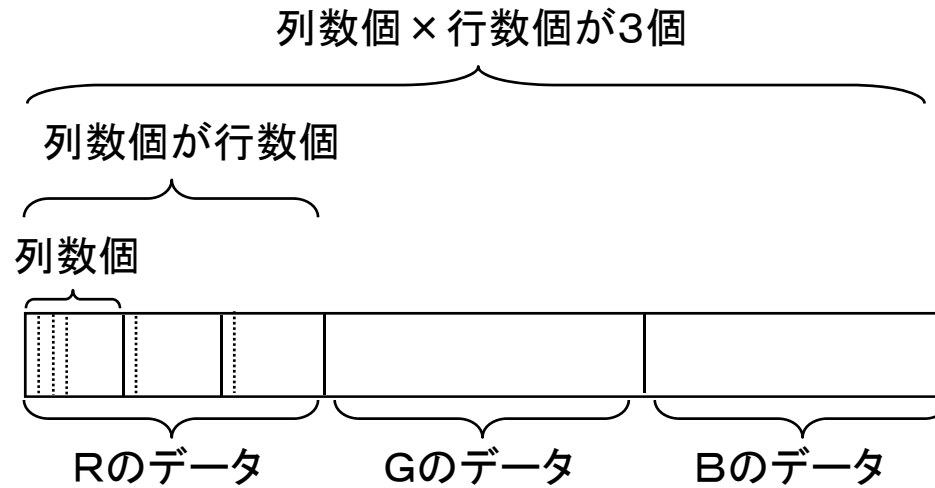
利点

- ・ループの終了判定(分岐命令)の回数が減る
- ・並列処理しやすくなる

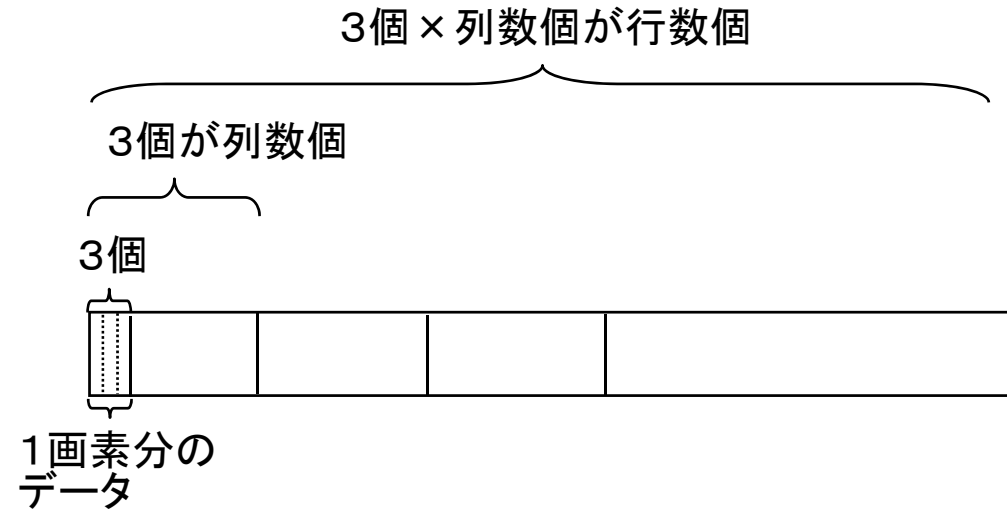
欠点

- ・ループ展開後のソースコードと実行ファイルのサイズ増大

画像データのメモリ内配置 (CSQ)

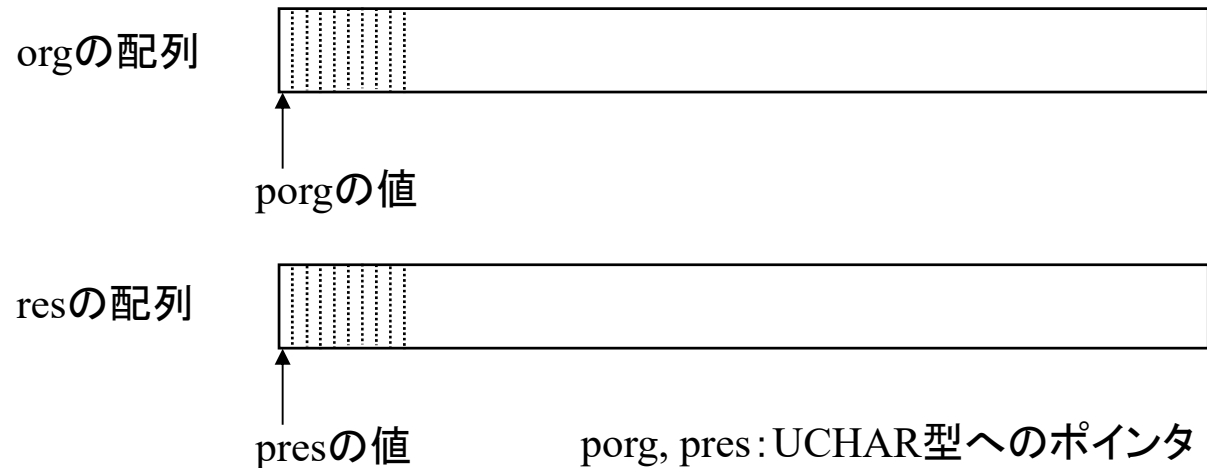


画像データのメモリ内配置(CIP)



ポインタを利用したコピー

- 画像データがメモリ内に連続的にとられていることを利用したコピー方法(3次元配列を1次元配列とみなす)



$*pres = (*porg) / 2$ で1個分のデータが2で割ってコピーされる

presとporgをインクリメント(1データ分右に移動)し、次のデータをコピー

コピー時間測定実験(3) kadai3-3.c

▲課題: ポインタ利用コピーの場合について、実験(1)と同様にしてコピー時間を測定し、考察せよ。

- ・まず、ポインタを利用してコピーする関数copy3dimg_point(.....)を完成する.
- ・CSQ,CIPの何れか一方だけで良い
- ・消費時間の測定は必ず画像の出力をコメントアウトしてから行う.
- ・測定結果は両対数のグラフにし, 実験(1)(2)の結果と比較して考察する。